



このようにこぼれたデータは、たいていソフトウェアをクラッシュさせ、時には重大な結果を招くことになるのです。

こんな単純なミスを犯してしまったのはESAだけではありません。アメリカのNASAといえば、優秀な人材が集まっていることで知られています。当然、ソフトウェアの設計開発にも優秀な人材がそろっています^{※2}。しかし、いかに優れ者の集団といっても、バグからは逃れられない運命にあるようです。

覚えている方もいると思いますが、1999年に火星探査衛星がソフトウェアのトラブルのために落ちてしまいました。なぜだかご存知ですか？ きっとよほど複雑な宇宙での出来事か、それともブラックホールに吸い込まれたのかと思うかもしれません。

実はプログラム内部で使う単位を間違えた^{※3}のだそうです。ご存知かと思いますが、アメリカでは通常、距離の単位としてキロメートル

※2：その中には、「NASAはなんたってお役所だから休み取り放縦だよ」などと呑気なことを言ってるマイク君がいるので、ちょっと信用ならないのですが、まあ優れた研究はよくやっています。

※3：うちのかみさんがアメリカ製の体重計に乗って、「私がこんなに重いわけではない！」と叫んでいる状態と同じです（彼女はアメリカに何年か住んでいたのにいまだにパウンド（アメリカの重さの単位）とキログラムの換算ができません）。

の代わりにマイルを使っています。この単位系の違いは、私達のようにメートル単位に慣れている人間にとってこのうえなく不便なことです。筆者がアメリカに滞在しているとき、うちのかみさんのみならず結構な数の外国人がよく単位換算を間違えていました。そしてNASAも間違えてしまったのです。しかも火星で。

NASA曰く、「探査衛星がどこかへ行ってしまった主な原因は、アメリカで使われている単位系からメートル単位系への変換がタコだったためである」。うちのかみさんと同じ間違い?? やれやれ……。

その結果、火星探査衛星は軌道はずれて、どこかへ行っちゃいました。NASAの研究者にはコンピュータサイエンスの博士号を持ってる人もいます。それなのに、一体なぜこんなことが起きるのでしょうか?

このように、ソフトウェアの品質というのは非常に重要で、場合によってはその良し悪しで損害が非常に大きいことがわかっていただけだと思います。

大きな事故は報道されますが、小さいソフトウェアの問題はあまり知らされません。しかしどこの会社でも（少なくとも私が勤めた会社では）数億円もしくは数百億円という規模で損害をこうむっています（大きな声じゃ言えませんが）。

1-2 テスト担当者の心得

先人の言葉に学ぶソフトウェアテストの奥義

どんなに優秀な開発者であっても逃れられないソフトウェアの問題を防ぐ手立てとして、ソフトウェアテストはどのような役目を果たす

ものかを考えてみましょう。ここでは難しい説明は抜きにして、先人の言葉を借りることにします。

バグを全部見つけるのは無理だと心得よ。

————— by Cem Kaner

最初から責任を放棄しているわけではありませんが、ソフトウェアのすべてのバグをなくすことは不可能です。ソフトウェアは十分複雑な工業製品なのです。

**エラーは見つからないだろうという仮定のもとに
テストの計画を立ててはいけない。**

————— by G.J. Meyers

多くの人は自分の書いたプログラム、自分の関わっている製品にバグがなければいいという希望的観測を持って開発やテストをしています。そのため、ソフトウェアテストの作業でバグが見つかりづらくなり、出荷後ユーザからバグを指摘されることになるのです。

**プログラム開発グループは、自分たちのプログラムを
テストしてはいけない。**

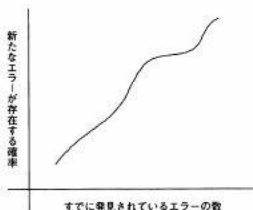
————— by G.J. Meyers

これは簡単なようですが実行に移すのは難しいでしょう。なぜなら日本の場合、ほとんどのテストが開発者自身によって行なわれているからです。しかし、どんな小さいプロジェクトでもテストをする人間とプログラムをする人間を分けるべきです。

例えば、あなたがあるソフトウェアを期日までに仕上げる仕事を任されたとしましょう。担当者はあなた1人だけで、プログラミングとテストをすべて担当することになったとします。たぶん、あなたはプログラムを設計し、コードを書き、コンパイラを通し、軽くそのプログラムを動かしてみるでしょう。そして次に何を考えるでしょう？「私の書いたコードにバグはない。テストも終わったし、飲みに行くことにしよう！」。

プログラムのある部分でエラーがまだ存在している確率は、
すでにその部分で見つかったエラーの数に比例する。

———— by G.J. Meyers



あんまりテストを知らない上司：

「うーん、60%のカバレッジじゃ足りないから80%まで持っていてほしいな」

カバレッジを完全に理解していない部下：

「そうですか、経験的にいうと80%は難しいんですね」

以前「いまさら聞けないソフトウェアテストの手法」というセミナーの講師をしたとき、満員の会場に品質管理業務に携わる管理職の方々もいたのですが、制御パステストについてよく知らない方も多かったようです。知らないけれども管理職をやってる人もいるんだと驚いたことがあります。確かにこの手法を知らなくても業務はやっていけます。しかし、知っていればもっと業務に幅が広がりますし、なにしろ心配の種が減ります。

ある大手の家電の品質保証部長（筆者の勤務先ではありません）は、製品ラインアップが複雑化したため、単純なテストケースだけでも膨大な量になり、製品を出すのが心配でしょうがない、そして品質に自信がないと語っておられました。制御パステストはエンジニアに自信を与えるテスト手法です。「この製品の品質とは」と聞かれたときに「80%のカバレッジを実現しています」と答えればいいわけですから。

大人気ゲームソフトのバグ

さて、本論に入る前に1つの例を紹介しましょう。1999年2月にファイナルファンタジーVIIIにバグが見つかり新聞に載るような騒ぎになりました。このバグは、ゲームのプレイ中に、以下の3つの条件が重なった場合に起こると伝えられました。

①ディスク3枚目で「トラビ
ア溪谷」をクリアする

②①の後に「セントラ遺跡」
での制限時間付イベントに
臨む

③「セントラ遺跡」内での敵
との戦闘中に制限時間に達
した際に、「やめる・ゲーム
オーバー」を選ばずに「遺
跡入口まで戻って再トライ
する」を選んでしまった場
合

フローチャートを見るとわ
かりますが、それほど複雑な
プログラムとは思えません。
もし、テスト担当者が制御バ
ステストを行なっていればこ
のような大きなバグは発売前
に見つけることが可能だった
かもしれません。制御バステ
ストによるコードカバレッジ
テストの本質はこのようなフ
ローチャートをちゃんとカバ
ーすることにあります。

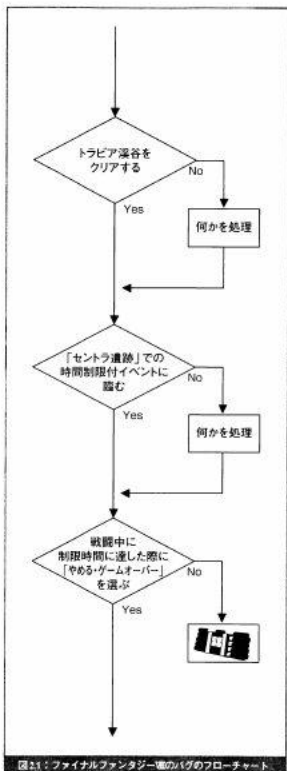


図21：ファイナルファンタジーVIIのバグのフローチャート

ステートメントカバレッジ

まずはじめに、制御バステストの中で一番簡単に理解できるステートメントカバレッジを説明します。ステートメントカバレッジではコードの中の命令文（ステートメント）を少なくとも1回は実行します。

以下のようなプログラムを考えてみましょう^{※6}。

```
if( con1 == 0){  
    x = x + 1;           //①  
}  
if ( con2 > 1){  
    x= x * 2;           //②  
}
```

このプログラムに対して、開発者なら①と②の部分をまずテストし、それでテスト終了と考える方が多いのではないのでしょうか？ その場合、テストケースは con1=0の場合と con2=2で命令文①②はテストできます。これがステートメント（命令文）カバレッジです。このプログラムのフローチャートを書いてみると図22のようになります。

しかし、図22を見ると、con1=0とcon2=2を入力したとしてもc、そしてeのパスを通るテストが抜け落ちていることがわかるでしょう。例えば、開発者がcon2>1と書くべきところをcon2>0と書き

※6：ここではCのコードを例としてあげています。テスト担当者はかならずしも複雑なプログラミング言語のスキルは必要ないと思いますが、プログラムの基本構造だけは学ぶことをお勧めします。CでもJavaでも書店で売っている一番簡単に薄い本を買って理解するだけで十分です。要は開発者と同じ言語で話すために必要なプログラミング言語知識だけは持っていたほうがいいということです。

てしまった場合のバグはステートメントカバレッジ手法では見つかりません。

このように、ステートメントカバレッジはまったく役に立たないとはいいいませんが、非常に弱いテスト手法です。プログラムによっては、ステートメント（命令文）よりも分岐でのエラーが非常に多い場合もありますし、かつ発見されにくい傾向にあります。

製品の例で考えてみると、会計用のソフトで出金や入金の額が違っていたら非常に問題になりますし、その金額演算のバグが発見された場合はその機能を書いた開発者は非常に恥ずかしい思いをします。例えば、商品の定価に消費税5%を上乘せするソフトウェア処理があった場合、計算によって商品の値段が5倍になってしまうコードを書いた開発者は同僚との飲み会でさんざ

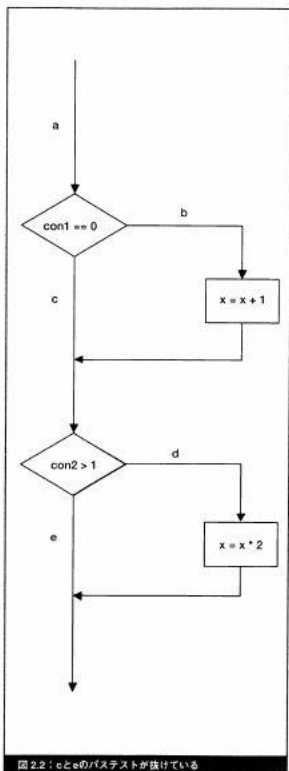


図 2.2 : cとeのパステストが抜けている

前項で紹介した制御バステストは、プログラムがどのような振る舞いをして、どのように制御され実行されていくかをテストするものでした。ここで取り上げるデータフローバステストでは、制御部分ではなくプログラム中で使われるデータに焦点を当ててテストします。

まずはデータ（変数等）とは一体どんなものなのかを考えていきましょう。プログラミング言語による多少の差異はありますが、データはおおまかにいえば3つの状態に分けられます [BE190]。

◎定義済

データは定義されているが、まだ使用されていない。

◎消滅

消滅または未定義。データは定義されたことがあるがその後解放されたり消滅したりしている。

◎使用

データが使用されている状態。例えば変数が左辺に現れデータが右辺より代入される状態^{*11}。

この定義を利用するといくつか明らかになってくることがあります。それは、データというのは開発者が自由に定義したり使ったりすることができるように思えますが、実は規則的にしか使えないということ

※11：少し説明が難しい部分です。プログラム言語によっては「定義」という概念が少し異なるのでBasicやFORTRANというより、CやJavaを想定して読んでいただいたほうが理解が早いと思います。

です。基本的には上記の3つの組み合わせしか存在せず、組み合わせによってはバグになります。

例えば、**a**という変数がまずはじめに定義され、**a**という変数に2が代入 ($a=2$) され、それが倍にされるようなプログラム ($a=2 \times a$) を考えてみましょう。ここでは、変数の扱いは定義済—使用—使用となり、バグはなく正常に変数が使用されたということになります。以下にすべての組み合わせをまとめてみました。

◎定義済—定義済

プログラム実行上実害はないが、なぜ2度も定義してるかは疑問の余地がある。

◎定義済—消滅

データが定義された後、使われずに消滅したことになる。バージョンアップが重なったソフトウェアに多く見られるもので、これはプログラムからの削除対象になる。

◎定義済—使用

普通のプログラムスタイル。データを定義してから正常に使用している。

◎消滅—消滅

2回消滅を繰り返す必要は当然ない。実害があるかないかはプログラミング言語によるがコードのバグであることは確かである。

◎消滅—使用

明らかなバグ。データが未定義、もしくは解放された後に使用される。多くの場合プログラムの制御は危機的状態に陥る。

◎使用－定義済

バグではないが意見の分かれるところである^{*12}。間違えではないが多くの開発者は違和感を覚えるであろう。その変数がどのような目的で作られ、正しく使われているかをチェックするほうが無難。

◎使用－消滅

通常のケース、問題はない。

◎使用－使用

通常のケース、問題はない。

データは上記のような組み合わせに対する属性を持ち、定義済一定義済など不適切な属性を持つことがあります。しかし、これを開発者もしくはテスト担当者が行なうのは単純作業であることを考えると時間があまりにもったいないですね。このようなバグを発見するには静的解析ツールなるものがあるのでそれを使用することをお勧めします^{*13}。

ただし、このような実害のないバグの扱いは注意しなければなりません。この種のバグは多くのいらぬ機能やコードを削るバージョンアップ時に多く出てきます。本来はプロジェクトの初期段階で削除作業をするべきです。これをプロジェクトの後期になって、いらぬコードだといって削除すると、想像もできない問題が出る場合があります。例えばコンパイラが期待していたコードを生成しなくなったり、予想もしないメモリのバグが出ることもあるので注意が必要です。

^{*12} Steve McConnellの著書『コードコンプリート』とBoris Beizerの『ソフトウェアテスト技法』では意見が異なります。おそらく開発者とテスト担当者というそれぞれの立場による意見の相違と思われます。

^{*13} 静的解析ツールとしては、Splint (<http://www.splint.org>) などがあります。

データフローバステスト（全使用法）

ここまでは、データを形式的に間違った使い方をしていないかをチェックするためのテストについて述べてきましたが、次はプログラム上でどのようにデータが使われるか、そしてそれをどのようにテストしていくのかを解説していきます。

残念ながらデータフローバステストもたくさんの手法があり、その中のどれを用いるのか議論の分かれるところでもあります（また、すべての方法を理解するのも難しいことです）。ここでは、全使用法（all-uses）というものを説明します^{※14}。全使用法は一番シンプルで使いやすい方法だと思います。

全使用法はすべての変数について、すべての定義からはじまり、変数を使用するすべてのパスをカバーします。例として次のコードを考えてみましょう。

```
if( con1 ==1){  
    x = 10;    //①  
}  
else{  
    x = 20;    //②  
}  
if ( con2 == 2){  
    y= x;      //③  
}  
else{  
    y = x * x; //④  
}
```

※14：データフローバステストの方法論についてもっと詳しく知りたい場合は、Boris Beizerの著書「ソフトウェアテスト技法」を参照してください。

そのために、まず入力値を2つの部分集合に分ける必要があります。1つは有効同値、もう1つは無効同値です。有効同値とはプログラムが期待する入力値です。今回の例では、入力A、Bともにプログラムは1から999が入力されることを期待しているので、有効同値は1から999です。無効同値はそれ以外のあらゆる値ということになります。以下の図を見るとわかりやすいでしょう。

無効同値 (-0)	有効同値 (1-999)	無効同値 (1000-)
-----------	--------------	--------------

テストケースを書いてみよう ～非常に強いテストケース～

では実際に有効同値と無効同値のテストケースを書いてみましょう。テストケースは有効同値と無効同値全体をしっかりとカバーできるように全部で10個用意しました。

<有効同値クラスのテストケース>

A = 300、B = 400

<無効同値クラスのテストケース>

A = -5、B = -20

A = 0、B = 400

A = -20、B = 1005

A = 500、B = -30

A = 600、B = 1200

A = 1100、B = -50

A = 1500、B = 70

A = 3050、B = 4500

A = 0、B = 0

ここでは有効同値をA = 300、B = 400としていますが、理論的にいえば、1から999の範囲であればどんな値でもかまいません。一般的には中央値付近の最もユーザが使いそうなデータを入力するのが多いようです。これらの値をグラフで示すと図3.3のようになります。

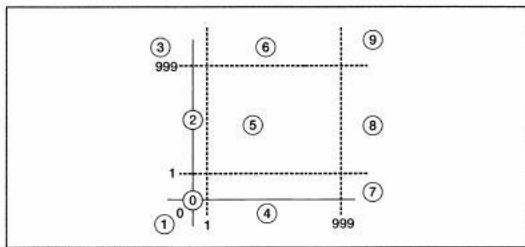


図 3.3：非常に強い同値クラステスト

ここで問題となるのが、有効同値が1つに対して無効同値が9つあることです。この例は非常に単純なので9つで済んでいますが、実際のプログラムはもっと複雑なものになります。したがって、無効同値のテストケースを作成して実行するのは非常に時間がかかります。そのためソフトウェアテストの現場では、ある程度この無効同値のテストを省略する必要があります。

省略するのはやむをえないのですが、その省略によって致命的なバグを発生させることは許されません。例えば、ある無効同値を入力した後、プログラムがフリーズしたり、保存されたデータが破壊されたりするのは避けなければなりません。賢明な読者はすでに矛盾点にお気づきかと思います。

テストケースを省略するのはやむをえないが、
その省略されたテストケースでバグを出してはいけない。

———— by Juichi Takahashi

格言のようにまとめてみましたが、この矛盾というか無理難題にテスト担当者は常に立ち向かわなければなりません。残念ながら、テスト担当者はベストを尽くしても報われないときがあるのですが。

さて、無効同値が9つもあると、普通の人ならテストケースを減らす方向で考えたくになります。次にその減らし方について考えていきましょう。

テストケースの数を減らすには ～実践的なテストケース～

9つの無効同値を使った「強い同値クラステスト」から無効同値を減らす「実践的な同値クラステスト」を考えることによってテストケースの数を減少させる方法ですが、これがちょっと難しい。Jorgensen [JOR02] は、強い同値クラステストの場合は1つのエレメント（最小分割できる同値クラス）に対し、1つのテストケースを割り当てています。「実践的な同値クラステスト」は1つの同値クラスに対し1つのテストケースを割り当てます。まとめると図3.4のようになります。

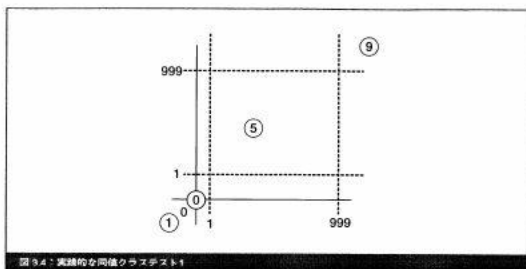


図 3.4：実践的な同値クラステスト

「実践的な同値クラステスト」※23の場合はテストケースは以下の4つです。

<有効同値クラスのテストケース>

$A = 300$ 、 $B = 400$

<無効同値クラスのテストケース>

$A = -5$ 、 $B = -20$

$A = 3050$ 、 $B = 4500$

$A = 0$ 、 $B = 0$ ※24

この場合①⑨を無効同値クラスとしましたが、図3.3で示す③⑦でもかまいません（図3.5）。

※23：原著ではWeak Robust Equivalent Test としていますが、本書ではそのWeak Robust Equivalent Testが実際にテストに関わるエンジニアが一番向いているテスト手法だと考え「実践的な同値クラステスト」と呼んでいます。

※24：ゼロは常に特別な値です。入力できる場合はかならずテストケースに入れてください。

これをプログラムとして書くと以下ようになります。

```
if( a > 0 && a <= 999)
    //正しい値が入力されたときの処理
else
    //間違った値が入力されたときの処理
```

これをディシジョンテーブルで示してみましょう。まず、ソフトウェアがどのような状態をとりうるのかを切り分ける必要があります。以下の4つの組み合わせがあります。

- ABとも入力正しい場合 (ルール1)
- Aの入力が正しくて、Bの入力が正しくない場合 (ルール2)
- Aの入力が正しくなくて、Bの入力が正しい場合 (ルール3)
- ABとも入力が正しくない場合 (ルール4)

これら入力の4つの状態に対して、出力は次の2つの状態をとりうるものと考えます。1つはABとも入力範囲の値が入力され、正しく計算されて出力される場合。もう1つはABのどちらか、もしくは両方の値が入力されソフトウェアが入力エラーを出力するといった場合です。これを表にまとめてみましょう (表3.2)。

この方法には問題が1つあります。表を使ったテストは非常に小さいソフトウェアか、あるいは大きなソフトウェアの一部分をテストするときにしか使うことができません。例えば1000を超えるような項目をテストするのにこの表を用いるのはかなり困難です。

表 3.2：ディシジョンテーブル

状 態	ルール			
	ルール1	ルール2	ルール3	ルール4
C1:A=正しい	T	T	F	F
C2:B=正しい	T	F	T	F
動 作				
A1:計算値出力	✓			
A2:入力エラー出力		✓	✓	✓

T:TRUE(真) F:FALSE(偽)

逆に項目が少なくかつ複雑な動きをするソフトウェアには非常に有効です。なぜなら、ある機能のテストがざっくり抜けることを防げるからです。7章の「7.3 テストケースの書き方」で紹介しているような方法でテストケースを書くと、どうしても分量が大きくなり、一目で全体のテストケースの状態を眺めることができません。しかし、表にまとめれば全体のテスト構成が見ることができ、テストケースの抜けを発見しやすくなります。

有限状態マシン (Finite State Machine) という用語は、たぶん多くの方が学校や職場で一度は見聞きした単語だと思います (もちろん覚えているかは別にして。筆者はテストを学んではじめて状態遷移について理解しました)。有限状態マシンはソフトウェアの設計から矛盾の検出などにも幅広く用いられます。ここでは有限状態マシンを利用した状態遷移テストを紹介します^{*31} [MAR95] [DEI90]。

状態遷移とは

ソフトウェアは一般的に常に同じ状態 (ステート)^{*32}にあるわけではなく、状態を変化させてユーザに対するソフトウェアの操作を容易にしています。例えばワープロソフトの場合、何もダイアログボックスを表示せずに、すべての機能をショートカットキーに割り当てるとしたら、非常に使いづらいソフトウェアになるでしょう。

それと同時にソフトウェアの設計も難しくなります。通常は、印刷ダイアログボックスを出している間は文章の編集ができないようにするなど、適宜ダイアログを表示させて機能を制限しています。もし、印刷ダイアログボックスが表示されているときに、さらに文書を変更できるようなワープロソフトを作ろうとしたら、変更を印刷に反映させるために大量にコードの追加が必要になるでしょう。同時に、それがユーザにとってよい機能といえるかは疑問です。

*31：別名『モデルベーステスト』とも呼ばれます。

*32：ディンジョンテーブルの「状態」とは概念が異なります。



図3.11：DOS コマンドラインインターフェース（コマンドプロンプト）

そのため、ある程度ユーザの操作に制限を加える必要が生じ、「状態 (state)」という概念が生み出されたのです。ソフトウェアの中には、状態の区別があまりはっきりしないものもあります。例えばDOSやLinuxのコマンドラインインターフェースは状態遷移テストを使用する利点がほとんどありません。なぜならいつでもすべてのコマンドを受け入れる状態になっているため、状態が1つしかないからです。

状態遷移テストとは、要は「状態」をモデル化してテストを行なう手法といえます。まず、状態遷移は、大きく分けて状態 (state) と遷移 (transition) の2つによって表現されます。以下の図に示すようにある状態から他の状態に移るには入力Xによる遷移が必要です。

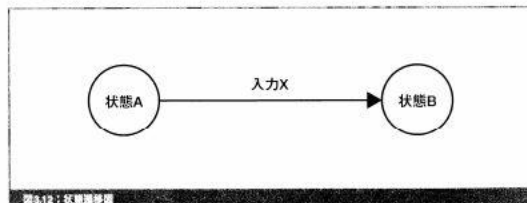


図3.12：状態遷移図

またWhittakerはこんなこともいっています。

もしデータの入力制限があれば、
他の方法でその制限を越える入力を行ないなさい。

Microsoft PowerPoint 2000を例に説明しましょう。

テーブルを挿入するときに、PowerPoint 2000で列と行の数をそれぞれ25と入力します。テーブルを挿入した後、さらに列や行を挿入してみると、明らかにPowerPointのレスポンスは遅くなり、最後にはフリーズのような状態に陥ります。

なぜこのような状態に陥るかという、プログラムがどうも25行までしか扱うことを想定していないためと思われます。テーブルを挿入する段階で試しに行と列の数をそれぞれ26行に設定するとよくわかります。PowerPointは自動的に26という数字を25に直し、26行（26列）以上のテーブルを作れないようにしているのです。

よいテスト担当者はバグの匂いをかけるといわれることがあります。しかし、凡人のテスト担当者は「あ、そうね、26行以上は入らないのね。じゃあテストケースにエラー処理のテストとして26行を入れるように書いておこう！」ということになります。

よいテスト担当者は「うむ、25行の制限が加えられているのなら、26行以上のテーブルを他の方法で作ることができればたちまちプログラムが不安定になるに違いない！」と思うはずです。そして25行を入れた後、どうにかして26行以上のテーブルを作ろうと努力するのです。

Whittakerは「テスト担当者はやぶ医者のごとし」と語っています。例えば、足をケガした人が病院に来た場面を考えてみましょう。

けが人「足をケガしたんですよ」

医 者「どれどれ、見せてください」（そのケガの部分を叩いてみる）

けが人「あ！ 痛いです」

医 者「そうですか、じゃあこれはどうですか？」（もっと強く叩いてみる）

けが人「痛いといってるじゃないですか。なんでケガの場所をそんなに強く叩くんですか！」

医 者「いや強く叩かないと、どのくらいひどいケガかわからないのでね」

ソフトウェアテストも同様に、少しでもプログラムの挙動が変な場所があったら、すかさずそこを攻撃するのが正しいテスト担当者の姿です。開発者に「わかったわかった、もうバグを見つけるの止めてくれ。バグの出ないようにデザインから変更するから」なんていわれるのがテスト担当者にとって最高の賛美の言葉なのです。

4

ソフトウェアの性能を チェックする

—— システムテスト

今までは主にプログラムの小さな部分、もしくは特定の機能にフォーカスするテスト手法を説明してきました。ここで紹介する「システムテスト」という手法は、どんな状況下であってもソフトウェアが期待された動作をするかをテストする手法です。

4-1

テストの工程によって手法を使い分ける

システムテストと機能テスト

システムテストは誤解されやすい、また機能テストと混同されやすいテスト手法です。異論はあるかと思いますが、本書では「機能テストでカバーできないテストをするテスト」をシステムテストと定義します。

ただ、システムテストと機能テストの境界線はあいまいで、重なり合う部分もあります。両者の違いは、テスト工程中のどの時期に実施するかで区別することができます。システムテストはテスト工程にお



では、各ステップについて説明していきましょう。

Step1：アーキテクチャバリデーション

まず、ソフトウェアがアーキテクチャ的に十分なパフォーマンスを発揮しているかをチェックします。例えば、データベースの場合、その処理能力が、要求仕様に合致しているかなどを机上でチェックします。もしくは非常に小さいプロトタイプソフトウェアを作って、実際に要求仕様で定義されたデータ量を処理できるかを測ります。

Step 2：パフォーマンスベンチマーク

実際に開発されたソフトウェアのテストを行ないます。パフォーマンステストができる状態のソフトウェアが準備できたらすぐやるのが基本です。

Step 3：パフォーマンス回帰テスト

回帰テストを行います^{*39}。このテストはプログラムが開発途中で常に変更されている状態で行ないます。変更したことによってパフォーマンス低下を招かないように定期的にチェックします。

Step 4：パフォーマンスチューニング、アクセプタンステスト

最終製品が要求定義に定められたパフォーマンスを出しているかをチェックします。もちろんパフォーマンスが何らかの原因で要求定義値以下であれば出荷はできません。

^{*39}：回帰テストについては、6章「その他のテスト手法」を参照してください。

Step 5: 24×7 パフォーマンスモニタ

ERPアプリケーションなどの場合特に必要になるでしょう。ERPアプリケーションを開発する場合、顧客のデータを完全に利用できないことがあります。その場合は顧客の使用するデータに近いダミーデータでテストします。データのサイズやデータのバリエーション^{※40}がなるべく同じになるようデータを作成しましょう。もし作成したデータが顧客の使用する実データとかけ離れていると、開発中に行なったパフォーマンステストでは出なかった問題が実運用上で起こる恐れがあります。

パフォーマンステストに関しても、負荷テストと同様、テストの自動化^{※41}は必須です。

4-5

「使いやすさ」について考える

ユーザビリティテスト

ユーザビリティテストとは、実際にプログラムを使うユーザもしくはそれに近い人に使用してもらい、ユーザインターフェースなどの使い勝手や仕様をチェックしてもらうテストです。10年前なら「(使いにくくても難しくても) ソフトウェアの操作を覚えてください」といえたかもしれませんが、現代のソフトウェアではそれはできません。ユーザインターフェースは使う人の要求をよく考慮に入れて設計されるべきです。

ユーザビリティテストは、ソフトウェアの機能をテストするのでは

※40：似通ったデータでテストを行なうと、キャッシュのヒット率が上がり、期待より高いパフォーマンスが出る場合があります。

※41：自動化テストについては9章「テストの自動化という悪魔」を参照してください。

なく、ユーザの側に立ってその要求を満たしているかを確認するテストになります。また、ユーザビリティテストの対象はソフトウェアだけでなく、システム全般におよびます。紙のマニュアル、オンラインヘルプもそれに含まれます。

ユーザビリティテストでチェックする要件は以下のようなものです。

●チェックポイント1：アクセサビリティ

十分簡単に操作できますか？ 文字が小さかったり、ボタンが小さかったりすることはありませんか？

●チェックポイント2：効率性

よく使う機能なのに、何回もマウスの操作を必要としていませんか？ また、あまり使わない機能が前面に出ていませんか^[HOW96]？



図 4.5：メニューバーがほとんど表示されている Microsoft Word の画面

例えば機能がたくさんあるソフトウェアでは、あまり使わない機能がある程度使いにくくする必要があります。図4.5のようにMicrosoft Wordがすべての機能をアイコンで表示したらどうでしょう？ 逆にユーザがよく使う「保存」や「プリント」の機能がどこにあるかわかりづらくなってしまいます。

●チェックポイント3：学習性

簡単にソフトウェアの使い方が理解できますか？^[FER01] ある程度使えばすべての機能が使えるようになりますか？ かなり使い込んでいても、操作方法がよくわからないソフトウェアもあります。

●チェックポイント4：構造の明確さ

ソフトウェアの構造がユーザにとってクリアですか？ ユーザが今どこを操作していて、何ができるかが理解しやすい設計になっていますか？ ヘルプ・ドキュメントが適切に書かれていますか？ ユーザが困った際にそれらが問題を解決してくれますか？

●チェックポイント5：操作性

ユーザが操作を誤ったときに、操作を支援する機能がありますか？^[SWE03]

●チェックポイント6：ユニバーサルデザインへの配慮

ユニバーサルデザイン^{※42}の観点から、視聴覚に障害を持つユーザ

※42：IBMの「バリアフリーの扉」というWebページに障害者、健常者双方に使いやすいデザインのガイドラインが載っています。<http://www-6.ibm.com/jp/accessibility/>

これは、入力した1つの文字を表示させる単純なプログラムです。以下に実行例を示します。「program_format」とコマンドを入力したあと、続けて「222」という数字を入力すると、「222」が表示されます。

```
C:¥>program_format 222
222
c:¥>
```

では、次のような文字列を入れてみましょう。

```
C:¥>program_format "%x %x %x %x"
0 135340 7ffdf000 ccccccc
c:¥>
```

あら不思議、変な数字が表示されました。実はこれはスタックの情報なのです。このようにprintfやsprintfなど書式付き文字列を扱う関数は正しく使わないと、スタック情報を書き換えられることがあります。テスト担当者は詳しい攻撃アルゴリズムを知る必要はありませんが、C言語でprintfを使うと非常に危険だということを認識すべきでしょう。

printfはC言語を習うときに一番最初に使う関数ですが、この場合はprintf(argv[1]);の代わりにprintf("%s", argv[1]);と書く必要があります。しかし、それだけでは不十分なのでしっかり入力チェックをしなければなりません。

残念ながらバッファオーバーフローやフォーマットバグ攻撃に対抗

するテスト方法は多くありませんし、かなり原始的な方法がまだまだ主流です。多くの研究者は境界をチェックする以外の方法はないと述べています [MCG01]。それでもテストしなければなりません。

続いて完璧とはいえませんがセキュリティ機能テストの手法をいくつか紹介しましょう。

5-3 プログラムレベルで攻撃を防ぐには セキュリティ機能テスト

本章ではセキュリティテストを「セキュリティ機能テスト」と「セキュリティシステムテスト」に分けて説明をします。本来セキュリティテストはシステムテストの一部なのですが、現代のソフトウェアではシステム上でセキュリティテストをただけでは十分ではありません。そのためこの「セキュリティ機能テスト」の項では、コードレビューから単体モジュールレベルのテスト手法まで、いくつかの手法を説明していきます。

その前にプログラミング言語の選択がセキュリティテストに与える影響の大きさについて触れておきましょう。もしプロジェクトチームがC/C++を選んだ場合はセキュリティテストチームはプロジェクトの終了まで長期休暇は取れないでしょう。なぜならテスト項目数があたかも無限大に発散してテストが終わらないからです。特にNXビットを搭載していないシステムでは状況はさらに困難を極めます。

実際にそんなプロジェクトをたくさん抱えている会社があります。Microsoftです。彼らは毎週のようにセキュリティパッチをリリースしています。彼らは決してなまけものではないですし、テスト費用を惜

しんでいるわけでもありません。世界中から頭のいいエンジニアが集まって努力しています。しかしそれでもこの状態です。

一方、C/C++の代わりにJavaを選んだ場合、セキュリティに関するテストタスクは著しく減ります※51 [NIK04]。Java言語がCに比べ実行速度が遅いことはよく知られており、その理由だけでC/C++言語が選ばれる傾向があります。しかしセキュリティということを考えた場合に、Javaを開発言語として選ぶことによってセキュリティ上の問題のかなりの部分が改善されることを無視するわけにはいきません。もし十分にCPUのパワーがあり、メモリが必要なだけ使える状態ならば、Javaをプログラミング言語として選ぶのは悪い選択ではないでしょう [NIK04]。

セキュリティの問題の重大さは、パフォーマンスの問題の比ではありません。パフォーマンスの問題の場合「ほんとこのシステム遅いんだよなー。次のバージョンでパフォーマンス改善してよ」というのがユーザのクレームですが、セキュリティ問題の場合「おい！ すぐ直せ！ 今直せ！」ということになるのは想像できると思います。

コードレビュー

コードレビューに関しては、本書ではほとんど記述してきませんでした。確かにテスト手法より効率よくバグが見つかるケースはありますが、たいていのバグはテストによって見つかると考えられます。しかし、セキュリティ機能のバグについては、テスト手法で見つけるのが明らかに困難な場合が多いので注意が必要です。「うちはきっちりし

※ 51：しかしJavaで書いたからといってセキュリティ問題が完全に起こらないとはいえないところがこの問題の難しいところです。Javaの「セキュリティコードガイドライン」は以下のURLを参照してください。
<http://java.sun.com/security/seccodeguide.html>

たテストチームがいるので」といってレビューをしない方針の会社もあるかもしれません。しかし、セキュリティ機能がある製品を出荷する場合はコードレビューを実施することをお勧めします。

入力テスト

境界値分析をして徹底的な入力テストをする必要があります。徹底的な入力テストだけが唯一確実なセキュリティテスト手法です。

この進んだ世の中にあってもセキュリティテストは地味に入力を繰り返すだけなのです。ただその中でも、より有効な入力テストの手段というものがあります。モジュール指向のテストなどもその1つです。続いてそれらの方法を紹介していきましょう。

モジュール指向のテスト

Howard ^[HOW02] がモジュール指向のセキュリティテストに関するプロセスを定義していますが、ここでは少し変形させたプロセスを用います。

1. アプリケーションを基本モジュールに分解する。
2. モジュールに対する入力を定義する。
3. その入力によってモジュールの脆弱性が露見するかどうかを判断する。
4. 脆弱性があると考えられるモジュールに様々なデータを入力する。

まあそれほどたいしたプロセスではありません。アプリケーションを基本コンポーネントに分解するという手法 ^[MYE80] はソフトウェアテストで昔から用いられているものです。テストの工程が増えるという意味

ストの場合、このアプローチは時間とコストがかかりすぎます。そこで、私は自分のチームにはサイドエフェクトが絶対存在すると仮定し、回帰テストを行なうときに以下のことを実行するように伝えています。バグを直したつもりで直っていなかったり、直したことによって不具合が生じることがあっては、テストの意味がありません。それを防ぐためにも回帰テストはしっかり行なう必要があるのです。

◎バグがちゃんと直っているか確認する（当然のことですね）

◎バグを直すことにより、他に関連するバグが発生しないかを確認する。

◎バグはプログラムのある一部分に固まっているので、どこかにバグが発見されたら、その機能のバグが起こりやすいという認識を持ち、隠れたバグを探す。

回帰テストの自動化

回帰テストは一般に自動化が容易とされています。確かにキャプチャー・リプレイ機能^{※62}を持つテスト自動化ツールを使えば簡単に回帰テストが実行できそうです。しかし多くの場合、回帰テストの自動化の作業は無駄に終わることを覚悟しなければなりません。

自動化プログラムを作成する場合、何も立ち上がってない状態からアプリケーションを起動し、問題のある機能のテストを行ないます。例えば、「アプリケーション立ち上げ」→「ファイルのロード」→「文

※62：多くの自動化テストツール、例えばMercury WinRunner、IBM Rational Robotなどはテスト担当者が1回操作すると、その操作を記憶して何度でも自動的に繰り返しテストが可能な機能（キャプチャー・リプレイ機能）が備わっています。

字の入力」などです。

しかし、問題の機能（バグの発生した機能）にいたる途中でユーザインターフェースに変更が発生した場合、自動化スクリプトがまったく走らなくなる場合があります※63。

さらにこの回帰テストはそれほど多く実行されるものではありません。残念ながら数回走らせればいい方です【KAN00】。したがって、自動化のメリットはあまりありません。回帰テストは、バグが開発者によって修正された直後と出荷直前に行なえば十分だからです。

また、回帰テストを実行する場合は、重要度の高い順序で行なうよう気をつけた方がよいでしょう※64【ROT03】。回帰テストは時間がかかるため、優先順位をつけずに実行すると思いのほか時間がかかってしまうからです（バグデータベースを読み返しても、数ヶ月前に報告されたバグは再現しにくいものです）。出荷間際の回帰テストは有効性が高いため行なうことが多いと思います。より重要なバグは早めに見つけたほうが出荷スケジュールに影響を与えないことはいうまでもありません。

※63：自動化に関しては、9章「テストの自動化という悪魔」を参照してください。

※64：優先順位は、バグの重要性、もしくは各モジュール内での代表的なバグなどを基準にして決めるとよいでしょう。

ここまでホワイトボックステスト、ブラックボックステスト、システムテストといったテスト手法を説明してきましたが、これらのテスト手法の中で、いったいどれを使った方がいいのかと思う方もいるでしょう。ここでは、テスト手法の取捨選択方法を紹介します。

一番よい方法は、今まで行なったテスト手法の中からよいものを選ぶことです。そのために前回のテストデータは残しておくべきでしょう。特にどのようなバグが発生したかを調査しておく、テスト手法の選択はしやすくなります。

では、はじめてテストする製品の場合はどうしたらよいでしょうか。その場合、表1のような表を使ってみるのも手だと思えます。この表を見ると「構造」と「データ」のバグが多く発生しています。構造に関してはカバレッジでかなりの部分がテストできると思います。やはりカバレッジデータを取ることが基本だということがわかりますね！

構造 Structural	25.18%
データ Data	22.44%
開発された機能の不良 Functionality as implemented	16.19%
インプリメンテーション Implementation	9.88%
システム統合 Integration	8.98%
要求定義 Functional requirements	8.12%
テスト定義、実行に関わる不良 Test definition or execution	2.76%
システム、ソフトウェアアーキテクチャー System, software architecture	1.74%
特定できず Unspecified	4.71%

表1：バグの発生原因

もう1つ有用と思われるデータ^[JON96]を表2に示します。ソフトウェアテストの範疇ですべてのバグを見つけ出すことは一般的に困難です。ある程度のバグは開発者が自分で見つけるべきでしょう。

本書ではコードインスペクションや、コードレビューなどはあまり取り上げていませんが、これからのテストエンジニアやQAエンジニアは、テスト技術だけでなく、インスペクション (inspection; 仕様やコードを机上でチェックして不具合や問題点を見つける技術) をどのように運用していくか、もしくはインスペクションとテストをどのように組み合わせるべきかという非常に難しい課題を扱わなければなりません。

QA活動の種類 Activity	レンジ
カジュアルなデザインレビュー Informal design review	25%-40%
フォーマルなデザインレビュー Formal design inspection	45%-65%
インフォーマルなコードレビュー Informal code reviews	20%-35%
フォーマルなコードインスペクション Formal code inspection	45%-70%
モデル化やプロトタイプ作成 Modeling and prototyping	35%-80%
個人的なコードチェック Personal desk-checking of code	20%-60%
ユニットテスト Unit test	15%-50%
新機能テスト New function (component) test	20%-35%
統合テスト Integration test	25%-40%
回帰テスト Regression test	15%-30%
システムテスト System test	25%-55%
小規模のベータテスト (10サイト以下) Low-volume beta test (<10 site)	25%-40%
大規模のベータテスト (1000サイト以上) High-volume beta test (> 1000)	60%-75%

■プロジェクトに悪影響を与える人員追加

大規模ソフトウェアの開発ではたいいてい1年くらいスケジュールが遅れることになっています。ということはほとんどのプロジェクトでは遅れを取り戻すため、もしくはこれ以上遅らせないために開発人員の追加をする可能性があります。

しかし、プロジェクトが遅れたからといって、人員をやみくもに増員するのはまったく賛成できません。テストはソフトウェア開発の最後の砦の作業です。その重要な作業を、プロジェクトをよく知らない新人に任せたりするのはどうかと思います。

ある日の出来事。筆者はWindowsのAPIをテストするテストコードをCで書いていました。そのときもテスト作業が遅れていたのが夜の10時になっても仕事をしていました。すると、友人の開発者が入ってきて、こんな会話になりました。

友人「大変だねーいつも。なんなら俺が手伝ってやろうか？」

私「悪いね。そしたらこのテストケース書いてくれる？」

友人「OK。でも俺の書いたテストコードって誰がテストするんだい？」

私「……」

もし増員して、その人が信用ならない人だったら？ その人はプロジェクトでの暗黙のルール（ドキュメントに書かれていないルールはたくさん存在するはずです）を知ってますか？ 開発者が書いたコードはテスト担当者がテストしますが、テスト担当者が書いたコードや、テストケースは誰がチェックすることになっていますか？ 残念ながらテ

ストにおける人員の追加は、開発における人員の追加より一般に困難で危険です。

**遅れの出ているソフトウェア開発プロジェクトに
人員を追加すると、さらに遅れる**

————— マーフィーの法則

マーフィーの法則には冗談も含まれていますが、真理も突いているようです。McConnelも同様のことを述べてます [MCC93]。

とはいえ、いくつかの特別な状況であればプロジェクトの最後の段階で人を追加することは有効です。条件としては新たに入る人がする仕事は独立しており、メインのプロジェクトと明らかに分離していることが必要です。

こんなたとえ話があります [MCC99]。「さあ、子供を1人産みましょう！ というゴールがあった場合、9人の女性がいくらがんばっても1人の女性と同じスピードでしか産めない。なぜなら9人の女性の間ではタスクがシェアできないからである」*72。要は明確に分離できる仕事の場合は増員により効率化することができるということです。

ただし、プロジェクトの遅れを増員することで取り戻す場合、次に挙げる負の要因をよく考慮に入れなければなりません。

*72：このたとえ話を作った Steve McConnel は世界有数のソフトウェア学者ですが、たまに比喩表現がうまくないときもあるのです。

◎増員した場合、その人がチームの一員として力を発揮するまでには時間がかかる。さらにプロジェクトが遅れている場合、往々にして困難かつ複雑な状況下にチームが置かれているが、新メンバーがそれらの状況を理解するには時間がかかる。

◎チームに新たにテスト担当者を加えた場合、そのテスト担当者が新しい技術の取得や環境に適應するのに時間がかかるだけではなく、既存のチームメンバーの時間も費やされる可能性がある。例えばプロジェクトの概要を説明し、チームメンバーを紹介し、扱いに気を付けるべき開発者（開発者のかなりの数が、かなり低いコミュニケーション能力しか持たない）を教えなければならない。しかし、もしそれらの教育の時間を取らなければ、彼はチームにとっての厄介者になってしまう。

■人員や時間をどう見積るか

テスト責任者は明確な予算策定とコストに対する意識を持つべきです。なぜなら品質とコストは相関関係にあるからです。しかしながら、テストにかかるコスト計算は難しいものです。スケジュールが延びればコストが増える上、不確定要素はスケジュールだけではないからです^{※73}。

さらに、どれだけの人員をテストにかけるかはプロジェクトの責任者の感覚に依存します。筆者は長い間テストの責任者をやってきましたが、残念ながらテストに対してしっかりしたコスト意識をもったプロジェクト責任者と働いたことはありません。多くの場合、前年ペー

※73：テストコストについては“Effective Methods for Software Testing” [PER95]に詳細に書かれているので参考にしてください。

スやら、他の既存のプロジェクトと似たような予算を与えられ、その中でテストチームを運営してきました。

では、テストのコストに最も影響するのは何かというと、当然ソフトウェアの品質です。極論すれば完全なソフトウェアを作ればテストなんていらないと言い続けて数十年、ソフトウェアの品質は変わってない気がします。

IBMの調査によると、平均的なソフトウェアのコード1000行のうちに60個のバグが存在するといわれています^[PER96]。もしこのデータを利用するとしたら、1万行のソフトウェアを開発する場合600個のバグが存在すると推定できます。1人のエンジニアが1日3つのバグを見つけると仮定すると、そのコストは次のように計算できます。

$$\begin{aligned} & 600\text{個} \div 3\text{個} \text{ (1日あたりのバグ発見数^{※74}) } \times \\ & 7\text{万円} \text{ (1日あたりのざっとした人件費) } = 1\text{千4百万円} \end{aligned}$$

うーん、高い！ でもそのぐらいかかっていると筆者は思います。このコストの内訳としては以下のものを想定しています。

- ◎バグを見つけるコスト
- ◎テストケースを作成および実行するコスト
- ◎修正されたバグを確認するコスト (回帰テスト)

※74：ここでは仮に3個としていますが、平均的なテスト担当者がどれだけのバグを発見できるかについての学術データは存在しません。またソフトウェアがPCのGUIアプリケーションなのか、ライブラリのAPIを提供するものなのかによって大きく違ってきます。そのため過去のプロジェクトを参考にしてこの値は変更してください。

テストプランの理想と現実

IEEE 829についてざっと説明しましたが、残念ながら筆者はIEEE 829テンプレート自体は「まあまあ」の内容だと思いますが、ベストとは思えません。IEEEやISOスタンダードすべてにいえることなのですが、あらゆるソフトウェアに適応しようとしているため、逆にどんなソフトウェアにも適応できない症候群に陥っている感もあります。読者にはたくさんのテストプランを見て、自分なりのテストプランを見つけることをお勧めします。

テストプランについてかなり長く書きましたが、かならずしも長大なテストプランを作成する必要はありません。テストプランは他部門、もしくは部下によって読まれるドキュメントであり、レビューが目的です。関係各所にこんなふうテストをするので、なにとぞよろしくと伝えるためのドキュメントといえるでしょう。

逆にいえば100ページ以上にもおよぶテストプランを皆でレビューしましょうというのは土台無理な話です。ドキュメントはちゃんとしたもの書くべきですが、「できるだけ」ちゃんと書くという程度にしたほうがよいと思います。一時期ソフトウェアプロセスが流行っていましたが（今も流行ってる？）、そこでは完全なドキュメントが要求されず。

CMMのレベル5を取ったソフトウェア会社は、きちんとしたドキュメントを書かねばなりません。しかしCMMレベル5を取った会社の製品の品質が本当に優れていますか？ もう少しその辺のこと考えながら作業をしてもいいのではないのでしょうか。

Tom DeMarcoは講演で「IBMはOS/2を作るにあたりドキュメント作成だけで数年間かかるといっていた。Microsoftはまったくドキュメ

ント作業なしに（言い過ぎだと思うが）、Windowsを開発した」と語っていました^[DEM04]。事実、筆者がMicrosoftのシアトル本社で働いていたときはまったくドキュメントなしに200人以上の開発者が爾々とプロジェクトを進めているのを見て驚いたものです（さすがにそれはやり過ぎじゃないかとも思いましたが）。

7-3

テストケースの書き方

効率的なテストケースの作成と管理

セミナーの講師などやっていると、「テストケースってどうやって書くんですか？」という質問によく出くわします。

「今までどうやってテストしてたんですか？」と聞くと、あまりちゃんとした答えが返って来ないので原因はよくわからないのですが、かなりの人がテストケースを書くのに自信がないようです。

テストケースの記述例

基本的にはテストケースはどう書いてもいいのです。ただ書くときにはあいまいな表現・記述をしない方がいいでしょう。テストケースは普通のテストサイクルで利用されるとともに、正しくバグ修正が行われたかをチェックする回帰テストにも使われます。開発期間中何度となく利用されるため、いつ、誰がやっても同じような結果が得られるように書かなければなりません。

例えばMicrosoft Notepadでファイルのセーブをテストするテストケースでは、以下のような記述が推奨されるでしょう。

テストケースID：NOTE1256

テストケースタイトル：ファイルの保存

実行ステップ：

ステップ1. NotepadをWindowsスタートメニューより
立ち上げる。

ステップ2. abcdefgとタイプする。

ステップ3. Ctrl+sキーを押す。

ステップ4. ファイル名入力エディットボックスに
C：¥text.txtと入力。

結果および確認：

ファイルが保存されたことを、ファイルマネージャで確認する
とともに、他のソフトウェア（DOSの“type”コマンド等）で入力内
容が正しく保存されていることを確認。

テストケース管理ツールを使う

テストケースの入力や管理のてっとり早い方法として、市販のテストケース管理ツールを導入するのも手だと思います^{*76}。日本ではこのようなテストケースを管理するツールはあまり浸透していませんが、アメリカでは盛んに使われています。

このようなツールを使うと次のようなメリットがあります。

*76： Empirix (<http://www.empirix.co.jp>)、e-Manager マーキュリー・インタラクティブ (<http://www.mercury.co.jp/>)、TestDirectorなどが日本で発売されています。

◎ツールを持ってテストケースを書くことによって、複数のテスト担当者が同じ粒度でテストケースを入力しやすい。

◎テストケースの管理がしやすい。

実はこのようなテストツールを使用するとテストマネージャの仕事量は激減します。テストマネージャは常にテスト担当者がどのくらいテストケースを実行しているか、そのうちいくつかのテストケースが失敗しているかを監視する必要があります。そのためにはこういった管理ツールは必須です。

筆者はテスト管理ツール以外のツールを使ってテストケースを管理するのはあまり好みません。Wordは100ページを超えるととたんに不安定になります。Excelは文章が打ちにくいようです。FileMakerも使ってみましたし、TeXも試してみましたがどれもうまくいきませんでした。

たしかに数百のテストケースを作成し、それを1回実行するだけなら、市販の安価なワープロソフトを使うことも可能ですが、数十回もテストケースを実行するプロジェクトなら、それなりにツールを使って管理しなければならないと思います。

テストケースはいくつ必要か

筆者は「いったいいくつのテストケースを作成すればいいのか」といった質問もよく受けますが、これはあまり答えたくありません……。

なぜならそれはとても難しい質問だからです。頼りの学者先生たちはあまりこうしたベーシックな研究をしていないようで、文献にも載っていないのが実情です。

この2つのポイントについて説明しましょう。

■私情・恣意の入らないものを選ぶ

残念ながら、データというものは都合のよいように（主に仕事を楽にする方向に）操作されがちです。

例えば筆者はメトリックスとして「テストケースの数」を使わないようにしています。もしテストケースの開発数をメトリックスにした場合、テスト担当者はただ単にテストケースを増やそうとするでしょう。そのテストケースはまったく無意味であるかもしれませんし、コードの10%しかテストしていないかもしれませんし、1件のバグも見つけれないかもしれません。そう、メトリックスデータがよくてもまったく品質の向上に寄与しない可能性があるのです。

逆に「バグの数」というのは開発者が真摯に品質のよいコードを書いたか否かの結果です。バグの発生する数を減らすことは簡単なズルでは実現できません。

■品質を十分代表するものを選ぶ

読者の皆さんはアメリカのプロバスケットボールの選手で、非常に個性的なことで知られるデニス・ロッドマン^{※78}を知っていますか？「デニス・ロッドマンはどういう選手だ」と聞かされると、NBAのファンは「暴力的である」「髪の毛が赤い」などと説明したものです。その説明だけを聞いたら人は彼をまるでヤクザみたいだと思うに違いありません。しかし、彼の本質は優れたバスケットボール選手です^[AOW97]。

※78：デニス・ロッドマンを清原選手に置き換えていただければ理解がより進むと思います。

何をいいたいかという点、ソフトウェアも同じで、表面的なもので評価するのではなく、その本質で評価する必要があるということです。

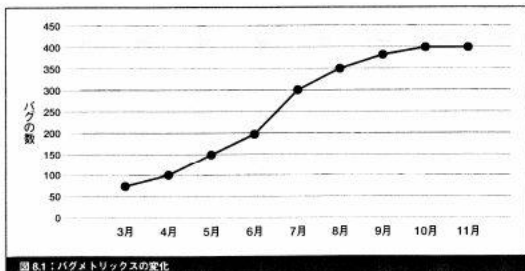
1つのメトリックス情報というのは氷山の一角でしかありません。ほんの一部分でソフトウェア全体を判断するのではなく、たくさんの細部を集めて氷山全体の大きさを測るようにしなければなりません。木を見て森を見ずにならないよう注意したいものです。

8-2

バグの数を管理する

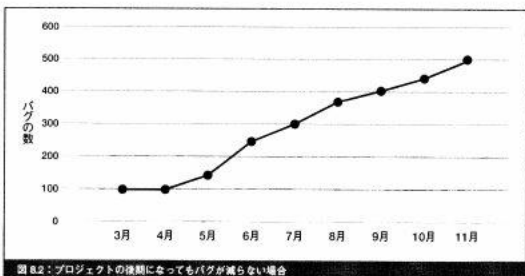
バグメトリックス

バグの数を管理することは、ソフトウェアテストにとって最も基本的なメトリックスです。あるプロジェクトのバグの数の推移をまとめた図8.1のグラフを見ると、開発の後期になるにしたがって、グラフの角度がなだらかになっています。このように、製品の出荷はバグの発見される数が少なくなってから行なわれることになります。

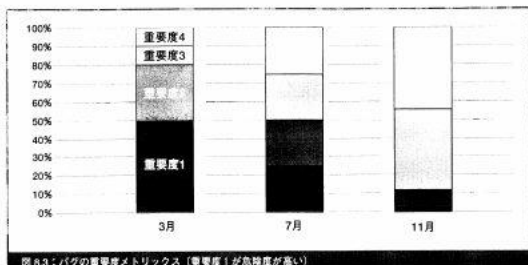


しかし、まれにバグの数が減らない場合があります。例えばGUIアプリケーションを開発しているときには、開発の最後の段階でオンラインヘルプやユーザインターフェースのチェックをすることがあります。その際、文字が少し欠けているとか、オンラインヘルプの文章がおかしいといった問題が断続的に出てきます。そういったプロジェクトの場合、図8.2のように開発後期になってもグラフの角度は急なままです。

一般的にテストが終わりに近づくと、重要度の高いバグ（ハングアップやフリーズ）の数は少なくなります（図8.3）。たとえバグの数が減っていないくても、重要度の高いバグが減っていれば品質が安定していると判断できます^{※79}。もちろん見つかるバグのうち、重要度の高いバグの比率が高いようなら製品の出荷はできません。



※79：筆者の関わった多くのプロジェクトでは、なかなか図8.1のようにはなりません。筆者が部下に対して「常にバグを見つけろ！」とか「バグを見つけられないテスト担当者はどこかへ行ってしまえ」とか「バグ1つ当たりいっただいお前に何百万払ってると思ってるんだ！」と叱咤激励、罵倒を繰り返した結果（成果？）です。でもこの場をかりてゴメンナサイ。



このように、単純にバグの発見数ではなく重要度の高いバグの発見数を用いて、ソフトウェアの品質を見極めることもできます。このメトリックスを使えば、ソフトウェアを出荷するか否かの判定を正しく行なうことができます。例えばバグの総数が減ってないからといって出荷時期を延期するようなことはなくなるでしょう。

メトリックスというのは、単に数字だけを取り上げて判断するのはなく、その数字の性質をよく知ったうえで利用すべきなのです。

バグの修正にかかる時間

バグメトリックスのもう1つの例は、バグの修正にどのくらい時間がかかるかというものです。一般的なプロジェクトでは、開発の初期段階で新しい機能の追加や、デザインの変更などがあるため、バグ修正以外の作業に時間がとられます。

その後、だんだん製品の仕様が固まるとともに新機能の追加が終了し、開発者のバグ修正時間が増えていき、バグ1個当たりの修正時間が